

GR Calculator

User Manual

Symbolic General Relativity Analysis
powered by SymPy

Edition: Desktop (`GR_python/`) & Google Colab (`GR_python_colab/`)
Language: Python 3.8+
Requires: SymPy, pdflatex

This manual covers every configuration option, metric example, and output section of the GR Calculator. No prior knowledge of the source code is required.

Contents

1	Introduction	3
2	Prerequisites & Installation	3
2.1	Python Packages	3
2.2	LaTeX / pdflatex	4
2.3	File Layout	4
3	Quick Start	5
3.1	Desktop Edition	5
3.2	Google Colab Edition	5
4	Configuration Reference	5
4.1	Coordinate Symbols (§1.1)	5
4.2	Metric Parameters (§1.2)	6
4.3	Metric Tensor <code>g_metric</code> (§1.3)	6
4.4	Inverse Metric <code>g_inv_metric</code> (§1.4)	7
4.5	Orthonormal Tetrad <code>e_tetrad</code> (§1.5)	7
4.6	LaTeX Substitutions <code>latex_subs</code> (§1.6)	8
4.7	Computation Flags (§1.7)	8
4.7.1	<code>COMPUTE_WEYL</code>	8
4.7.2	<code>COMPUTE_KRETSCHMANN</code>	9
4.7.3	<code>COMPUTE_GEODESICS</code>	9
4.7.4	<code>COMPUTE_KILLING</code>	9
4.7.5	<code>COMPUTE_TETRAD</code>	9
4.7.6	<code>FAST_MODE</code>	10
4.8	Output Options (§1.8)	10
5	Pre-Defined Metric Examples	10
5.1	Schwarzschild (Exterior Vacuum)	10
5.2	Minkowski (Spherical Coordinates)	11
5.3	FRW (Flat Cosmological)	11
5.4	General Static Spherically Symmetric	12
5.5	PG-Type Warp Metric (Areal Gauge)	12
5.6	PG-Type Warp Metric (Spatial Conformal) — Default	12
6	Custom Metrics	13
7	Understanding the PDF Report	14
8	Running the Calculator	15
8.1	Recommended: <code>gr_calculator.py</code>	15
8.2	Direct Execution	15
8.3	Calling <code>run_computations()</code> Programmatically	15
8.4	Calling <code>assemble_report()</code> and <code>write_and_compile_pdf()</code>	16
9	Google Colab Edition	17
9.1	Differences from the Desktop Edition	17
9.2	Notebook Workflow (8 Cells)	17
9.3	Colab-Specific Flags	17
9.3.1	<code>USE_PARALLEL</code> and <code>N_PARALLEL_JOBS</code>	17

9.3.2	USE_GPU	18
9.4	gr_numerics.py — API Reference	18
9.4.1	Backend Detection	18
9.4.2	Lambdification	18
9.4.3	Grid Evaluation	18
9.4.4	Visualisation	19
9.4.5	Geodesic Integration	19
9.5	Geodesic Integration — Schwarzschild Example	20
10	Advanced Topics	20
10.1	Tetrad Computation Details	20
10.2	Energy Conditions	21
10.3	Killing Vectors and Conserved Quantities	21
10.4	Weyl Tensor	21
10.5	Bianchi Identity Check	22
11	Performance Guide	22
12	Troubleshooting	23
A	Complete Configuration Flag Reference	23
B	run_computations() Parameter Reference	24
C	gr_numerics.py Quick Reference (Colab Edition)	24
D	Coordinate LaTeX Map	25

1 Introduction

The **GR Calculator** is a Python tool that takes a spacetime metric $g_{\mu\nu}$ as input and symbolically computes all standard quantities of General Relativity, then assembles the results into a formatted PDF report.

Computed Quantities

- **Metric** — covariant $g_{\mu\nu}$, contravariant $g^{\mu\nu}$, and determinant $\det g$
- **Christoffel symbols** — $\Gamma^\lambda_{\mu\nu}$ (connection coefficients)
- **Riemann curvature tensor** — $R^\rho_{\sigma\mu\nu}$
- **Ricci tensor and Ricci scalar** — $R_{\mu\nu}$, R
- **Einstein tensor** — $G_{\mu\nu} = R_{\mu\nu} - \frac{1}{2}g_{\mu\nu}R$
- **Kretschmann scalar** — $K = R^{\mu\nu\rho\sigma}R_{\mu\nu\rho\sigma}$ (curvature invariant)
- **Weyl conformal tensor** — $C_{\rho\sigma\mu\nu}$ (trace-free part of Riemann, 4D only)
- **Orthonormal tetrad** — e^μ_a and its verification $g_{\mu\nu}e^\mu_a e^\nu_b = \eta_{ab}$
- **Einstein tensor in orthonormal frame** — $\hat{G}_{ab}$
- **Energy conditions** — null (NEC), weak (WEC), strong (SEC)
- **Geodesic equations** — $\frac{d^2 x^\mu}{d\lambda^2} + \Gamma^\mu_{\alpha\beta} \frac{dx^\alpha}{d\lambda} \frac{dx^\beta}{d\lambda} = 0$
- **Killing/cyclic coordinates** — conserved momenta
- **Bianchi identity check** — $\nabla_\mu G^{\mu\nu} = 0$
- **Ricci trace check** — consistency of the scalar contraction

Two Editions

Feature	Desktop (GR_python/)	Colab (GR_python_colab/)
Platform	Any OS	Google Colab (Linux)
Entry point	<code>python gr_calculator.py</code>	<code>GR_Colab.ipynb</code>
Parallel computation	No	Yes (<code>USE_PARALLEL</code>)
GPU acceleration	No	Yes (<code>USE_GPU</code>)
Numerical evaluation	No	Yes (<code>gr_numerics.py</code>)
Geodesic integration	No	Yes (<code>diffraction / scipy</code>)

2 Prerequisites & Installation

2.1 Python Packages

Install the required packages with pip:

```
pip install sympy matplotlib scipy
```

- **sympy** (required) — symbolic mathematics engine
- **matplotlib** — required for the Colab edition plots
- **scipy** — required for geodesic integration fallback

For the Colab edition with GPU support (optional):

```
# CuPy -- NVIDIA CUDA GPU acceleration (fastest):
pip install cupy-cuda12x

# JAX + diffrax -- JIT-compiled geodesic integration:
pip install "jax[cuda12]" diffrax
```

Note

GPU packages are optional. If neither CuPy nor JAX is installed the numerical cells in the Colab notebook fall back to plain NumPy / SciPy automatically.

2.2 LaTeX / pdflatex

The calculator calls `pdflatex` to compile the PDF report. Install a TeX distribution appropriate for your operating system:

OS	How to install
Windows	MiKTeX — https://miktex.org/download
Linux	<code>sudo apt install texlive-full</code>
macOS	MacTeX — https://tug.org/mactex/
Any (online)	Overleaf — https://www.overleaf.com (upload the .tex file)

If `pdflatex` is not found the calculator still writes the `.tex` file and prints manual compilation instructions to the console.

2.3 File Layout

Desktop Edition (GR_python/)

```
GR_python/
gr_calculator.py  # Entry point -- run this file
gr_main.py        # User configuration, pipeline, PDF compiler
gr_tensors.py     # All symbolic GR computations
gr_latex.py       # LaTeX report assembly
gr_report.tex     # Generated after first run
gr_report.pdf     # Generated after first run (requires pdflatex)
```

Colab Edition (GR_python_colab/)

```
GR_python_colab/
GR_Colab.ipynb    # Main notebook -- open in Google Colab
gr_main.py        # Adapted from desktop (COLAB_MODE, parallel,
GPU flags)
gr_tensors.py     # Adapted (adds parallel computation functions)
gr_latex.py       # Identical to desktop edition
gr_numerics.py    # NEW: numerical evaluation, GPU, geodesic
integration
README_colab.md   # Quick-start guide
```

3 Quick Start

3.1 Desktop Edition

1. Open `gr_main.py` in any text editor.
2. In **Section 1.3** (around line 80), find the *Schwarzschild* metric block and remove the comment characters (`#`):

```

1 M = symbols('M', positive=True)
2 f = 1 - 2*M/r
3 g_metric = Matrix([
4     [-f,      0,      0,      0],
5     [ 0,    1/f,      0,      0],
6     [ 0,      0,    r**2,      0],
7     [ 0,      0,      0,  r**2 * sin(theta)**2],
8 ])
9 METRIC_NAME      = "Schwarzschild"
10 METRIC_DESCRIPTION = "Exterior vacuum solution"

```

3. For your first run, enable fast mode to save time:

```

1 FAST_MODE = True # skip Weyl and Kretschmann

```

4. Run the calculator from a terminal in the `GR_python/` folder:

```
python gr_calculator.py
```

5. Open the generated `gr_report.pdf`. Console output shows progress messages with timestamps.

Tip

For a complete run (including Weyl and Kretschmann) with the Schwarzschild metric expect approximately 5–15 minutes. Set `FAST_MODE = False` only when you want the full invariant analysis.

3.2 Google Colab Edition

1. Upload `GR_Colab.ipynb` to Google Colab (*File* → *Upload notebook*).
2. Run **Cell 1** to install LaTeX and Python packages.
3. Run **Cell 2**. When prompted, upload the four `.py` files from `GR_python_colab/`. The cell prints the detected GPU backend.
4. Edit **Cell 3** to select your metric and flags (see [Section 4](#)).
5. Run cells 4–8 in order.

4 Configuration Reference

All user configuration lives in `gr_main.py`, in the block labelled **Section 1**. Each sub-section below corresponds to a numbered part of that block.

4.1 Coordinate Symbols (§1.1)

```

1 # --- 1.1 COORDINATE SYMBOLS
   -----
2 t, r, theta, phi = symbols('t r theta phi', real=True)

```

```

3 coords = [t, r, theta, phi]
4 dim     = 4

```

coords A Python list of SymPy Symbol objects. The *order* must match the rows and columns of `g_metric`. The default is $\{t, r, \theta, \varphi\}$ for 4D spherical coordinates.

dim Integer spacetime dimension. Default 4. All tensor loops run over `range(dim)`. Use 3 for 2+1 gravity (note: Weyl is automatically skipped for $\text{dim} \neq 4$).

SymPy assumptions. Passing `real=True` when creating symbols helps SymPy produce cleaner simplifications. For parameters that are known to be positive (e.g. mass M , radius r) use `positive=True`:

```

1 M = symbols('M', positive=True)
2 r = symbols('r', positive=True)

```

Note

Any valid SymPy symbol name works for coordinates. Common choices: $\{t, x, y, z\}$ for Cartesian, $\{t, r, \theta, \varphi\}$ for spherical, $\{t, r, \chi, \psi\}$ for hyperspherical. LaTeX labels are pre-mapped for `theta`→ θ , `phi`→ φ , `chi`→ χ , `psi`→ ψ . Any other name prints as-is.

4.2 Metric Parameters (§1.2)

Define symbolic parameters and functions used inside the metric matrix.

```

1 # --- 1.2 METRIC PARAMETERS
  -----
2 M = symbols('M', positive=True)           # mass
3 # a = symbols('a', positive=True)         # spin parameter (Kerr)
4 # Lambda = symbols('Lambda')             # cosmological constant
5 # a_t = Function('a', positive=True)(t)   # scale factor a(t) for
  FRW
6 # A_r = Function('A')(r)                  # arbitrary radial
  function

```

Symbol vs. Function

Use `symbols()` for constant parameters (e.g. mass M , charge Q , cosmological constant Λ).

Use `Function('f')(r)` for functions of coordinates (e.g. $f(r)$, $a(t)$). The symbolic derivative of a `Function` is computed automatically by SymPy when Christoffel symbols are differentiated.

Assumptions

`positive=True` tells SymPy the symbol is positive, enabling cancellations like $\sqrt{r^2} \rightarrow r$ (instead of $|r|$).

4.3 Metric Tensor `g_metric` (§1.3)

```

1 # --- 1.3 METRIC TENSOR g_{mu nu}
  -----
2 g_metric = Matrix([
3     [g_tt, g_tr,  0,  0],
4     [g_tr, g_rr,  0,  0],
5     [ 0,    0, g_thth, 0],

```

```

6     [ 0,    0,    0, g_phph],
7 ])
8 METRIC_NAME      = "My Metric"
9 METRIC_DESCRIPTION = "Short description for the PDF header"

```

g_metric A $\text{dim} \times \text{dim}$ SymPy Matrix. Entry $[i,j]$ is $g_{x^i x^j}$ (covariant, lower-index metric).

Signature convention

The calculator uses the mostly-plus signature $(-, +, +, +)$. The sign of g_{tt} should be negative for a Lorentzian spacetime.

METRIC_NAME

Display string used in the PDF title and progress messages.

METRIC_DESCRIPTION

One-line description included in the PDF abstract.

4.4 Inverse Metric g_inv_metric (§1.4)

```

1 # --- 1.4 INVERSE METRIC g^{mu nu}
  -----
2 g_inv_metric = None # None = auto-compute via .inv() (recommended)

```

None The calculator calls `g_metric.inv()` automatically and applies `cancel()` to each entry. This is the recommended setting.

Manual matrix

Supply a $\text{dim} \times \text{dim}$ Matrix to skip the inversion step. Useful for large diagonal metrics where `.inv()` is slow but the inverse is obvious:

```

1 # Schwarzschild inverse (faster than auto-compute):
2 g_inv_metric = Matrix([
3     [-1/f, 0, 0, 0],
4     [0, f, 0, 0],
5     [0, 0, 1/r**2, 0],
6     [0, 0, 0, 1/(r**2 * sin(theta)**2)]
7 ])

```

4.5 Orthonormal Tetrad e_tetrad (§1.5)

```

1 # --- 1.5 ORTHONORMAL TETRAD e^{mu}_a
  -----
2 e_tetrad = None # None = auto-compute if COMPUTE_TETRAD = True

```

The tetrad e^μ_a (also written $e^{(a)}_\mu$) is a set of four orthonormal vector fields satisfying:

$$g_{\mu\nu} e^\mu_a e^\nu_b = \eta_{ab} \quad \text{where} \quad \eta = \text{diag}(-1, +1, +1, +1).$$

None + COMPUTE_TETRAD = True

The tetrad is computed automatically via ADM decomposition (see Section 10.1 for the three cases).

None + COMPUTE_TETRAD = False

Tetrad computation is skipped entirely; Sections 8–9 of the PDF (orthonormal frame, energy conditions) are omitted.

User-supplied Matrix

Always takes priority over auto-compute. Example for Schwarzschild:

```
1 e_tetrad = Matrix([
2     [1/sqrt(f),      0,      0,      0      ],
3     [      0, sqrt(f),      0,      0      ],
4     [      0,      0, 1/r,      0      ],
5     [      0,      0,      0, 1/(r*sin(theta)) ],
6 ])
```

4.6 LaTeX Substitutions latex_subs (§1.6)

```
1 # --- 1.6 LATEX SUBSTITUTIONS
  -----
2 latex_subs = {
3     r'\theta': r'\theta',      # identity example (no change)
4     # r'2 M': r'2M',          # remove space between coefficient and
      symbol
5 }
```

latex_subs is a Python dict mapping raw LaTeX strings to replacement strings. These replacements are applied as simple string substitutions *after* SymPy's latex() conversion and *before* the PDF is assembled.

Use cases:

- Remove unwanted spaces: r'2 M': r'2M'
- Replace a verbose function name with a compact symbol: r'B(r)': r'B(r)'
- Add custom notation for a parameter

4.7 Computation Flags (§1.7)

```
1 # --- 1.7 COMPUTATION FLAGS
  -----
2 COMPUTE_WEYL          = True
3 COMPUTE_KRETSCHMANN  = True
4 COMPUTE_GEODESICS    = True
5 COMPUTE_KILLING       = True
6 COMPUTE_TETRAD       = True
7 FAST_MODE            = False
```

4.7.1 COMPUTE_WEYL

Default	Cost	Restriction
True	High (minutes)	4-dimensional spacetimes only

Computes the **Weyl conformal tensor** $C_{\rho\sigma\mu\nu}$ (all indices lowered). The Weyl tensor is the trace-free part of the Riemann tensor:

$$C_{\rho\sigma\mu\nu} = R_{\rho\sigma\mu\nu} - \frac{2}{n-2}(g_{\rho[\mu}R_{\nu]\sigma} - g_{\sigma[\mu}R_{\nu]\rho}) + \frac{2}{(n-1)(n-2)}Rg_{\rho[\mu}g_{\nu]\sigma}.$$

Physical meaning: Weyl represents pure gravitational field (tidal forces, gravitational waves). It vanishes for conformally flat spacetimes (e.g., FRW cosmology). In vacuum, $C_{\rho\sigma\mu\nu} = R_{\rho\sigma\mu\nu}$.

Automatically skipped if $\dim \neq 4$.

4.7.2 COMPUTE_KRETSCHMANN

Default	Cost	Typical time
True	Very high	5–60 minutes

Computes the **Kretschmann scalar**:

$$K = R^{\mu\nu\rho\sigma} R_{\mu\nu\rho\sigma}.$$

This is a curvature invariant that diverges at genuine (non-coordinate) singularities. It remains finite at coordinate singularities such as the Schwarzschild horizon $r = 2M$.

Example (Schwarzschild): $K = 48M^2/r^6$

Warning

Computing K symbolically for metrics with abstract functions (e.g. $f(r)$, $B(r)$, $\beta(r)$) can take 10–60 minutes. Set `COMPUTE_KRETSCHMANN = False` or `FAST_MODE = True` for initial exploratory runs.

4.7.3 COMPUTE_GEODESICS

Default	Cost
True	Low (< 5 s)

Constructs the **geodesic equations** directly from the Christoffel symbols:

$$\frac{d^2 x^\mu}{d\lambda^2} + \Gamma^\mu_{\alpha\beta} \frac{dx^\alpha}{d\lambda} \frac{dx^\beta}{d\lambda} = 0.$$

The output uses abstract velocity symbols $\dot{x}^\mu = dx^\mu/d\lambda$ (for example `dot_r`, `dot_theta`).

4.7.4 COMPUTE_KILLING

Default	Cost
True	Low (< 1 s)

Identifies **cyclic (Killing) coordinates**: a coordinate x^μ is cyclic if

$$\frac{\partial g_{\alpha\beta}}{\partial x^\mu} = 0 \quad \forall \alpha, \beta.$$

For each cyclic coordinate the PDF lists the corresponding conserved momentum $p_\mu = g_{\mu\nu} \dot{x}^\nu = \text{const}$.

4.7.5 COMPUTE_TETRAD

Default	Cost
True	Medium

Enables automatic orthonormal tetrad computation via ADM decomposition. The tetrad is used to produce the Einstein tensor in the orthonormal frame and to evaluate energy conditions. A user-supplied `e_tetrad` always overrides this flag.

4.7.6 FAST_MODE

Default	Effect
False	Skips Weyl & Kretschmann

Convenient shortcut for exploratory runs. When `True`, both `COMPUTE_WEYL` and `COMPUTE_KRETSCHMANN` are overridden to `False` regardless of their individual settings.

Tip

Recommended workflow:

1. First run: `FAST_MODE = True` — quick results, all zero-curvature checks.
2. Second run: `FAST_MODE = False` — full invariant analysis.

4.8 Output Options (§1.8)

```

1 # --- 1.8 OUTPUT / DISPLAY OPTIONS
  -----
2 OUTPUT_FILENAME = "gr_report"      # base name for .tex and .pdf
3 AUTHOR_NAME     = "SymPy GR Engine" # author field in PDF metadata

```

OUTPUT_FILENAME

Base filename (no extension). The calculator writes `<OUTPUT_FILENAME>.tex` and compiles `<OUTPUT_FILENAME>.pdf` in the same directory as `gr_calculator.py`. In the Colab edition the output goes to `/content/`.

AUTHOR_NAME String placed in the PDF author field and report header.

5 Pre-Defined Metric Examples

Six metric templates are provided as commented-out code blocks in `gr_main.py` Section 1.3. To activate one, remove the `#` characters from that block and comment out all other `g_metric =` assignments.

5.1 Schwarzschild (Exterior Vacuum)

$$ds^2 = -\left(1 - \frac{2M}{r}\right) dt^2 + \left(1 - \frac{2M}{r}\right)^{-1} dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```

1 M = symbols('M', positive=True)
2 f = 1 - 2*M/r
3 g_metric = Matrix([
4     [-f, 0, 0, 0],
5     [0, 1/f, 0, 0],
6     [0, 0, r**2, 0],
7     [0, 0, 0, r**2 * sin(theta)**2],
8 ])
9 METRIC_NAME = "Schwarzschild"

```

```
10 METRIC_DESCRIPTION = "Exterior vacuum solution for a spherically
    symmetric mass M"
```

Expected results:

- Ricci tensor: $R_{\mu\nu} = 0$ (vacuum solution)
- Ricci scalar: $R = 0$
- Einstein tensor: $G_{\mu\nu} = 0$
- Kretschmann: $K = 48M^2/r^6$ (diverges at $r = 0$, finite at $r = 2M$)
- Bianchi identity check: all components = 0
- Cyclic coordinates: t and φ (energy and angular momentum)

5.2 Minkowski (Spherical Coordinates)

$$ds^2 = -dt^2 + dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```
1 g_metric = Matrix([
2     [-1,  0,  0,  0],
3     [ 0,  1,  0,  0],
4     [ 0,  0, r**2,  0],
5     [ 0,  0,  0, r**2 * sin(theta)**2],
6 ])
7 METRIC_NAME      = "Minkowski (spherical)"
8 METRIC_DESCRIPTION = "Flat Minkowski spacetime in spherical
    coordinates"
```

Use this metric to verify your setup. All curvature tensors must be identically zero. Non-zero Christoffel symbols arise from the coordinate system (not from curvature): $\Gamma^r_{\theta\theta} = -r$, $\Gamma^{\theta}_{r\theta} = 1/r$, etc.

5.3 FRW (Flat Cosmological)

$$ds^2 = -dt^2 + a(t)^2 [dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2]$$

```
1 a_t = Function('a', positive=True)(t)
2 g_metric = Matrix([
3     [-1,  0,  0,  0],
4     [ 0, a_t**2,  0,  0],
5     [ 0,  0, a_t**2*r**2,  0],
6     [ 0,  0,  0, a_t**2*r**2*sin(theta)**2],
7 ])
8 METRIC_NAME      = "FRW (flat)"
9 METRIC_DESCRIPTION = "Flat FLRW cosmological metric with scale factor
    a(t)"
```

Expected results:

- Weyl tensor: $C_{\rho\sigma\mu\nu} = 0$ (conformally flat)
- Ricci scalar: proportional to $\ddot{a}/a$ and $(\dot{a}/a)^2$

- Killing coordinates: r, θ, φ (spatial isotropy)

5.4 General Static Spherically Symmetric

$$ds^2 = -e^{2A(r)} dt^2 + e^{2B(r)} dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```

1 A_r = Function('A')(r)
2 B_r = Function('B')(r)
3 g_metric = Matrix([
4     [-exp(2*A_r),      0,      0,      0],
5     [      0, exp(2*B_r),      0,      0],
6     [      0,      0, r**2,      0],
7     [      0,      0,      0, r**2*sin(theta)**2],
8 ])
9 METRIC_NAME = "General static spherical"
10 METRIC_DESCRIPTION = "Static spherically symmetric metric with A(r), B(r)"

```

The Schwarzschild and Reissner-Nordström metrics are special cases. Results are expressed in terms of $A'(r)$, $A''(r)$, $B'(r)$.

Note

Computing Kretschmann for this metric is very expensive due to the symbolic function derivatives. Set `FAST_MODE = True` for a first run.

5.5 PG-Type Warp Metric (Areal Gauge)

$$ds^2 = -(1 - B^2\beta^2) dt^2 + 2B^2\beta dt dr + B^2 dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```

1 B_func = Function('B', positive=True)(r)
2 beta_func = Function('beta')(r)
3 g_metric = Matrix([
4     [-(1 - B_func**2 * beta_func**2), B_func**2 * beta_func, 0, 0],
5     [B_func**2 * beta_func, B_func**2, 0, 0],
6     [0, 0, r**2, 0],
7     [0, 0, 0, r**2 * sin(theta)**2],
8 ])
9 METRIC_NAME = "PG-type warp metric (areal gauge)"
10 METRIC_DESCRIPTION = "Painleve-Gullstrand-type metric"

```

This is a Painlevé-Gullstrand (PG) form: lapse $N = 1$, radial shift $\beta(r)$, spatial metric uses the areal radius. Used in warp-drive and analogue gravity analyses.

5.6 PG-Type Warp Metric (Spatial Conformal) — Default

$$ds^2 = -(1 - B^2\beta^2) dt^2 + 2B^2\beta dt dr + B^2 dr^2 + B^2 r^2 d\theta^2 + B^2 r^2 \sin^2\theta d\varphi^2$$

```

1 B_func = Function('B', positive=True)(r)
2 beta_func = Function('beta')(r)
3 g_metric = Matrix([
4     [-(1 - B_func**2 * beta_func**2), B_func**2 * beta_func,
5     0, 0],

```

```

5      [ B_func**2 * beta_func,      B_func**2,
6      [ 0,      0, 0],      0,      B_func**2 * r**2,
7      [ 0,      0],      0,      0,
8      [ 0,      0,      B_func**2 * r**2 * sin(
9      theta)**2],
10 ] )
METRIC_NAME      = "PG-type warp metric (Spatial Conformal)"

```

This is the **default active metric** in the unmodified `gr_main.py`. It includes a conformal factor B^2 on the spatial metric, commonly used in warp-metric analyses where $B(r)$ represents a radial distortion.

6 Custom Metrics

To add a metric not in the pre-defined list:

1. **Define coordinate symbols** (Section 1.1).
2. **Define parameters** (Section 1.2).
3. **Construct the matrix** (Section 1.3).
4. **Set names** (METRIC_NAME, METRIC_DESCRIPTION).
5. Optionally supply `g_inv_metric` or `e_tetrad`.

Example: Reissner-Nordström (Charged Black Hole)

$$ds^2 = -\left(1 - \frac{2M}{r} + \frac{Q^2}{r^2}\right) dt^2 + \left(1 - \frac{2M}{r} + \frac{Q^2}{r^2}\right)^{-1} dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```

1 M, Q = symbols('M Q', positive=True)
2 f_rn = 1 - 2*M/r + Q**2/r**2
3 g_metric = Matrix([
4     [-f_rn,      0,      0,      0],
5     [      0, 1/f_rn,      0,      0],
6     [      0,      0,  r**2,      0],
7     [      0,      0,      0, r**2 * sin(theta)**2],
8 ])
9 METRIC_NAME      = "Reissner-Nordstrom"
10 METRIC_DESCRIPTION = "Exterior electrovacuum solution, charge Q, mass
    M"

```

Example: de Sitter (Cosmological Constant)

$$ds^2 = -\left(1 - \frac{\Lambda r^2}{3}\right) dt^2 + \left(1 - \frac{\Lambda r^2}{3}\right)^{-1} dr^2 + r^2 d\theta^2 + r^2 \sin^2\theta d\varphi^2$$

```

1 Lambda = symbols('Lambda', positive=True)
2 f_ds = 1 - Lambda * r**2 / 3
3 g_metric = Matrix([
4     [-f_ds,      0,      0,      0],
5     [      0, 1/f_ds,      0,      0],
6     [      0,      0,  r**2,      0],

```

```

7     [ 0, 0, 0, r**2 * sin(theta)**2 ],
8 ])
9 METRIC_NAME = "de Sitter"

```

Tips for Custom Metrics

- Use `positive=True` assumptions on radial coordinates and scale factors to avoid spurious absolute values in output.
- For diagonal metrics, supply `g_inv_metric` manually to skip the (slow) generic matrix inversion.
- Apply `cancel()` inside the metric entries when components involve complex fractions. This pre-simplifies the starting point for Christoffel computation.
- Start with `FAST_MODE = True` to verify the basic curvature structure before running the full computation.

7 Understanding the PDF Report

The generated PDF is organised into the following sections.

#	Section title	What it contains
1	Abstract	Metric name, description, dimension, coordinates
2	Metric Tensor and Conventions	$g_{\mu\nu}$, $g^{\mu\nu}$, $\det g$, index conventions
3	Christoffel Symbols	Non-zero $\Gamma^\lambda_{\mu\nu}$ (zero components omitted)
4	Riemann Curvature Tensor	Non-zero $R^\rho_{\sigma\mu\nu}$
5	Ricci Tensor	Full matrix $R_{\mu\nu}$
6	Ricci Scalar	Expression for R
7	Einstein Tensor	Full matrix $G_{\mu\nu}$
8	Curvature Invariants	Kretschmann K (if enabled), Weyl components (if enabled)
9	Orthonormal Frame	Tetrad e^μ_a , detection method, orthonormality verification
10	Energy Conditions	ρ , pressures p_i , NEC, WEC, SEC expressions
11	Geodesic Equations	$\ddot{x}^\mu = -\Gamma^\mu_{\alpha\beta}\dot{x}^\alpha\dot{x}^\beta$
12	Conservation & Consistency	Bianchi $\nabla_\mu G^{\mu\nu}$, trace check

Interpreting the Bianchi and Trace Checks

Section 12 prints expressions that should all equal zero for a self-consistent computation:

- **Bianchi identity:** $\nabla_\mu G^{\mu\nu} = 0$ is printed for each ν . The calculator applies `cancel()` and then (if needed) `simplify()`. A result of 0 confirms the computation is self-consistent.
- **Trace check:** Verifies $G^\mu_\mu = -R$ (for the mostly-plus convention).

Note

A non-zero Bianchi result for metrics with abstract functions (e.g. $f(r)$) does not always indicate an error — SymPy may be unable to fully simplify the expression symbolically. Substitute numerical values for the functions and their derivatives to verify numerically.

8 Running the Calculator

8.1 Recommended: `gr_calculator.py`

```
cd GR_python
python gr_calculator.py
```

`gr_calculator.py` is a thin dispatcher that calls `gr_main.main()`. It adds the project directory to the Python path automatically, so it works regardless of the working directory.

8.2 Direct Execution

```
python gr_main.py
```

Equivalent to the above.

8.3 Calling `run_computations()` Programmatically

You can import and call the pipeline directly from your own script:

```
1 import sys
2 sys.path.insert(0, '/path/to/GR_python')
3 from gr_main import run_computations
4 from gr_tensors import symbols, Function, Matrix, sin
5
6 t, r, theta, phi = symbols('t r theta phi', real=True)
7 M = symbols('M', positive=True)
8 f = 1 - 2*M/r
9 g = Matrix([[ -f, 0, 0, 0], [0, 1/f, 0, 0], [0, 0, r**2, 0],
10             [0, 0, 0, r**2*sin(theta)**2]])
11
12 results = run_computations(
13     g_metric           = g,
14     coords             = [t, r, theta, phi],
15     dim               = 4,
16     g_inv_metric       = None,
17     e_tetrad           = None,
18     compute_weyl_flag  = True,
19     compute_kretschmann_flag = True,
20     compute_geodesics_flag = True,
21     compute_killing_flag = True,
22     compute_tetrad_flag = True,
23     fast_mode          = False,
24 )
25
26 print("Ricci scalar:", results['R_scalar'])
27 print("Kretschmann:", results['K'])
```


`run_computations()` Return Dictionary

Key	Type	Contents
'g'	Matrix	Covariant metric $g_{\mu\nu}$
'ginv'	Matrix	Contravariant metric $g^{\mu\nu}$
'det_g'	expr	Determinant $\det g$
'Gamma'	list[list[list]]	Christoffel $\Gamma^\lambda_{\mu\nu}$
'R_riem'	list ⁴	Riemann $R^\rho_{\sigma\mu\nu}$
'Ric'	Matrix	Ricci tensor $R_{\mu\nu}$
'R_scalar'	expr	Ricci scalar R
'G'	Matrix	Einstein tensor $G_{\mu\nu}$
'K'	expr / None	Kretschmann scalar K
'Weyl'	list ⁴ / None	Weyl tensor $C_{\rho\sigma\mu\nu}$
'e_tetrad_computed'	Matrix / None	Computed tetrad
'e_cov'	Matrix / None	Coframe
'tetrad_method'	str	'diagonal', 'adm_shift...', 'user_supplied'
'tetrad_verified'	bool	Orthonormality passed?
'tetrad_residual'	Matrix	$g_{\mu\nu}e^\mu_a e^\nu_b - \eta_{ab}$
'G_ortho'	Matrix / None	$\hat{G}_{ab}$ in orthonormal frame
'energy_conditions'	dict / None	ρ , p_i , NEC, WEC, SEC
'geodesics'	list / None	Geodesic equations
'killing'	list	Cyclic coordinate indices
'bianchi'	list	$\nabla_\mu G^{\mu\nu}$ (one entry per ν)
'trace_check'	expr	$G^\mu_\mu + R$ (should be 0)

8.4 Calling `assemble_report()` and `write_and_compile_pdf()`

For custom pipelines you can call the report functions separately:

```

1 from gr_latex import assemble_report
2 from gr_main import write_and_compile_pdf
3
4 report_lines = assemble_report(
5     results          = results,
6     coords           = [t, r, theta, phi],
7     dim              = 4,
8     metric_name      = "Schwarzschild",
9     description      = "Exterior vacuum",
10    author            = "My Name",
11    latex_subs_dict   = {},
12    g_metric_orig     = g,
13    e_tetrad          = None,
14 )
15
```

```

16 write_and_compile_pdf(report_lines, "/path/to/output/my_report")
17 # Produces: my_report.tex and my_report.pdf

```

9 Google Colab Edition

9.1 Differences from the Desktop Edition

File	Desktop	Colab
gr_latex.py	unchanged	identical copy
gr_tensors.py	core version	+ parallel helpers
gr_main.py	original	+ COLAB_MODE, parallel, GPU flags
gr_numerics.py	absent	new module
GR_Colab.ipynb	absent	8-cell notebook

9.2 Notebook Workflow (8 Cells)

Cell	Title	Action
1	Install dependencies	LaTeX, sympy, scipy; optional GPU packages
2	Upload & import modules	Upload .py files, detect backend
3	User input (edit this)	Set metric, coordinates, all flags
4	Symbolic computation	Call grm.run_computations()
5	Generate PDF	Assemble report, call write_and_compile_pdf()
6	Numerical evaluation	grn.evaluate_results_numerical()
7	Visualisation	grn.plot_gr_quantities()
8	Geodesic integration	grn.integrate_geodesic_jax()

9.3 Colab-Specific Flags

These three flags appear only in the Colab edition of gr_main.py:

```

1 USE_PARALLEL      = False    # parallel Christoffel + Riemann
2 N_PARALLEL_JOBS   = 2        # worker count (Colab free tier: 2 cores)
3 USE_GPU            = False    # CuPy/JAX for numerical post-processing

```

9.3.1 USE_PARALLEL and N_PARALLEL_JOBS

Distributes the Christoffel and Riemann computations across multiple CPU cores using Python's `ProcessPoolExecutor`.

Platform restriction

Linux / macOS only. Do *not* set `USE_PARALLEL = True` on Windows. Windows uses the 'spawn' process start method, which cannot serialise SymPy expression objects across process boundaries, causing a `PicklingError`.

Setting	Effect
<code>USE_PARALLEL = False</code>	Sequential (default; safe on all platforms)
<code>USE_PARALLEL = True</code>	Parallel; uses <code>N_PARALLEL_JOBS</code> workers
<code>N_PARALLEL_JOBS = 2</code>	Google Colab free tier (2 CPU cores)
<code>N_PARALLEL_JOBS = 4</code>	Google Colab Pro / standard Linux

9.3.2 USE_GPU

Tells `gr_numerics` to use CuPy or JAX when evaluating symbolic expressions numerically (cells 6–8). The symbolic computation in cell 4 always runs on CPU regardless.

Backend	Install	When used
CuPy (CUDA)	<code>pip install cupy-cuda12x</code>	GPU runtime selected in Colab
JAX (XLA)	<code>pip install "jax[cuda12]"</code>	If CuPy not available
NumPy (CPU)	already installed	Fallback if no GPU

9.4 gr_numerics.py — API Reference**9.4.1 Backend Detection**

```
1 grn.detect_backend()      # returns 'cupy', 'jax', or 'numpy'
2 grn.get_backend(name)    # returns the array module
```

9.4.2 Lambdification

Convert SymPy expressions to fast numerical functions:

```
1 fn = grn.lambdify_scalar(results['R_scalar'], coords, backend='numpy')
2 # fn(t_val, r_val, theta_val, phi_val) -> float or array
3
4 fns = grn.lambdify_matrix(results['Ric'], coords, backend='numpy')
5 # fns[i][j](t_val, r_val, ...) -> float or array
```

9.4.3 Grid Evaluation

```
1 import numpy as np
2
3 coord_ranges = {
4     t: (0.0, 10.0),
5     r: (2.5, 20.0),
6     theta: (0.1, 3.0),
7     phi: (0.0, 6.28),
```

```

8 }
9
10 num_results = grn.evaluate_results_numerical(
11     results, coords, coord_ranges, npts=40
12 )
13 # Returns dict: {'R_scalar': array, 'K': array}

```

Parameters:

coord_ranges Dict mapping each symbol to a (min, max) tuple. Coordinates absent from the dict default to (0, 1).

npts Grid points per axis. Memory usage scales as $npts^{\text{dim}}$, so keep this low for 4D metrics ($npts=20-40$).

9.4.4 Visualisation

```

1 # Plot R_scalar and K as 2D heat maps in the (r, theta) plane:
2 grn.plot_gr_quantities(num_results, coords, slice_axes=(1, 2))

```

`slice_axes=(i, j)` selects which two coordinate axes form the x- and y-axes of the plot. Central slices are taken along all remaining axes.

9.4.5 Geodesic Integration

```

1 import numpy as np
2
3 def rhs(tau, y):
4     """
5     RHS of first-order geodesic ODE.
6     y = [x0, x1, x2, x3, v0, v1, v2, v3]
7     Returns dy/dtau.
8     """
9     positions = y[:4]
10    velocities = y[4:]
11    # Replace zeros with lambdified Christoffel terms:
12    accelerations = [0.0, 0.0, 0.0, 0.0]
13    return list(velocities) + accelerations
14
15 y0 = [0.0, 6.0, np.pi/2, 0.0,      # initial position
16       1.0, 0.0, 0.0, 0.1]         # initial velocity
17
18 tau_arr, y_arr = grn.integrate_geodesic_jax(
19     rhs_fn = rhs,
20     y0      = y0,
21     tau_span = (0.0, 50.0),
22     n_steps  = 500,
23 )
24
25 grn.plot_geodesic(tau_arr, y_arr, coords)

```

The function tries **difffrax** (JAX-JIT) first; falls back to **scipy.integrate.odeint** if JAX or difffrax is not installed.

9.5 Geodesic Integration — Schwarzschild Example

For a Schwarzschild equatorial circular orbit at $r = 6M$ (with $M = 1$):

```

1 import numpy as np
2 from sympy import lambdify
3
4 M_val = 1.0
5 r0     = 6.0
6 f0     = 1.0 - 2.0 * M_val / r0
7 Omega  = np.sqrt(M_val / r0**3)
8
9 # Build RHS by lambdifying Christoffel symbols
10 Gamma = results['Gamma']
11 coords_num = [t, r, theta, phi]
12
13 # Lambdify all non-zero Christoffel components
14 Gamma_fn = {}
15 for lam in range(4):
16     for mu in range(4):
17         for nu in range(4):
18             expr = Gamma[lam][mu][nu]
19             if expr != 0:
20                 Gamma_fn[(lam, mu, nu)] = lambdify(
21                     coords_num, expr.subs(M, M_val), 'numpy'
22                 )
23
24 def rhs_schwarz(tau, y):
25     x = y[:4]
26     v = y[4:]
27     accel = [0.0, 0.0, 0.0, 0.0]
28     for (lam, mu, nu), fn in Gamma_fn.items():
29         g_val = fn(*x)
30         accel[lam] -= float(g_val) * v[mu] * v[nu]
31     return list(v) + accel
32
33 y0 = [0.0, r0, np.pi/2, 0.0,
34        1.0/f0, 0.0, 0.0, Omega/f0]
35
36 tau_arr, y_arr = grn.integrate_geodesic_jax(rhs_schwarz, y0,
37                                             tau_span=(0, 2*np.pi/Omega), n_steps=500)
38 grn.plot_geodesic(tau_arr, y_arr, coords_num)

```

10 Advanced Topics

10.1 Tetrad Computation Details

When `COMPUTE_TETRAD = True` and no tetrad is supplied, the calculator automatically constructs the orthonormal tetrad using one of three methods:

Case A: Diagonal metric

All off-diagonal entries of $g_{\mu\nu}$ are zero. Tetrad components: $e^\mu_a = \delta^\mu_a / \sqrt{|g_{\mu\mu}|}$.

Case B: ADM metric, diagonal spatial block

The metric has a non-zero shift vector (e.g. $g_{tr} \neq 0$) but the spatial part γ_{ij} ($i, j \in \{1, 2, 3\}$) is diagonal. An analytic coframe formula is used.

Case C: ADM metric, non-diagonal spatial block

Most general case; requires Cholesky factorisation of γ_{ij} to extract the spatial triad.

If Cholesky fails (e.g. the spatial metric is not positive-definite), the calculator prints a warning. In that case supply the tetrad manually.

Verification: After computing e^μ_a , the calculator checks $g_{\mu\nu}e^\mu_a e^\nu_b = \eta_{ab}$ and reports the residual matrix. A passed verification is printed as **TETRAD VERIFIED: OK**.

10.2 Energy Conditions

Energy conditions are extracted from the orthonormal-frame Einstein tensor $\hat{G}_{ab}$ via the Einstein field equations $G_{\mu\nu} = 8\pi T_{\mu\nu}$:

$$\begin{aligned}\rho &= \hat{G}_{00}/(8\pi) && \text{(energy density)} \\ p_i &= \hat{G}_{ii}/(8\pi) && \text{(principal pressures)}\end{aligned}$$

The conditions reported (as symbolic expressions) are:

Name	Abbreviation	Condition	Physical meaning
Null Energy Condition	NEC	$\rho + p_i \geq 0$	Non-negative null energy
Weak Energy Condition	WEC	$\rho \geq 0$ and NEC	Non-negative energy density
Strong Energy Condition	SEC	$\rho + \sum_i p_i \geq 0$	Gravity is attractive

Note

The symbolic output gives *expressions*, not verified inequalities. To determine whether a condition holds, substitute numerical values for the metric functions and parameters into the expressions.

10.3 Killing Vectors and Conserved Quantities

A coordinate x^μ is **cyclic** if the metric is independent of it:

$$\frac{\partial g_{\alpha\beta}}{\partial x^\mu} = 0 \quad \forall \alpha, \beta.$$

Each cyclic coordinate generates a Killing vector $\xi^\mu = \delta^\mu_a$ and a corresponding conserved quantity along geodesics:

$$p_\mu = g_{\mu\nu} \frac{dx^\nu}{d\lambda} = \text{const.}$$

Examples:

- In Schwarzschild: t cyclic $\Rightarrow$ conserved energy E ; φ cyclic $\Rightarrow$ conserved angular momentum L .
- In FRW: r, θ, φ cyclic $\Rightarrow$ spatial isotropy (but momenta not conserved due to expansion).

10.4 Weyl Tensor

The Weyl tensor $C_{\rho\sigma\mu\nu}$ is the totally trace-free part of the Riemann tensor. It vanishes if and only if the spacetime is **conformally flat**.

- **Vacuum** ($R_{\mu\nu} = 0$): $C_{\rho\sigma\mu\nu} = R_{\rho\sigma\mu\nu}$.
- **FRW cosmology**: $C = 0$ (conformally flat).
- **Schwarzschild**: $C \neq 0$ (tidal forces, $K = 48M^2/r^6$).

Physically, the Weyl tensor controls tidal deformation and gravitational wave propagation. It is automatically skipped when $\dim \neq 4$.

10.5 Bianchi Identity Check

The *contracted Bianchi identity* $\nabla_\mu G^{\mu\nu} = 0$ is computed for each $\nu = 0, \dots, \dim - 1$:

$$\nabla_\mu G^{\mu\nu} = \partial_\mu G^{\mu\nu} + \Gamma^\mu_{\mu\lambda} G^{\lambda\nu} + \Gamma^\nu_{\mu\lambda} G^{\mu\lambda}.$$

The calculator applies `cancel()` and (if needed) `simplify()`.

Interpretation:

- 0 — self-consistent computation.
- Non-zero *symbolic* expression — SymPy could not simplify fully. Try evaluating numerically.
- Non-zero *number* — indicates a programming error (report as a bug).

11 Performance Guide

Quantity	Computational cost	Typical time	Best practice
Inverse metric	Low	< 1 s	Supply manually for diagonal metrics
Christoffel $\Gamma^\lambda_{\mu\nu}$	Medium	5–30 s	Parallel on Colab (<code>USE_PARALLEL</code>)
Riemann $R^\rho_{\sigma\mu\nu}$	High	30 s–5 min	Parallel on Colab
Ricci, Einstein	Medium	< 1 min	Always computed
Kretschmann K	Very high	5–60 min	Disable for first run
Weyl $C_{\rho\sigma\mu\nu}$	High	2–20 min	Disable for first run
Geodesic equations	Low	< 5 s	Negligible cost

Recommendations

1. **First run:** set `FAST_MODE = True`.
2. **Supply `g_inv_metric`** manually for diagonal metrics to skip the symbolic matrix inversion.
3. **Use `positive=True`** on coordinate symbols and parameters to enable SymPy simplifications.
4. **Parallel computation** (`USE_PARALLEL = True`) on Colab or Linux: typically $2\text{--}3\times$ speedup for the Christoffel and Riemann phases on a 4-core machine.
5. **Simple metrics** (Schwarzschild, Minkowski): full run including Weyl/Kretschmann completes in 5–15 minutes.
6. **Abstract-function metrics** (e.g. $f(r)$, $B(r)$, $\beta(r)$): expect hours for Kretschmann — consider disabling it.

12 Troubleshooting

Symptom	Cause	Fix
<code>NameError: name '__file__' is not defined</code>	Script imported inside a Jupyter notebook	Use the Colab edition (<code>GR_python_colab/</code>)
PDF not generated; <code>pdflatex</code> message	<code>pdflatex</code> not in system PATH	Install MiKTeX / TeX Live; or compile manually
<code>PicklingError</code> or <code>AttributeError</code> with <code>USE_PARALLEL</code>	Windows spawn method cannot pickle SymPy	Set <code>USE_PARALLEL = False</code> on Windows
Computation runs for hours	Kretschmann / Weyl for abstract metrics	Set <code>FAST_MODE = True</code> ; disable <code>COMPUTE_KRETSCHMANN</code>
Non-zero Bianchi result	SymPy incomplete simplification	Usually harmless; substitute numbers to verify
Tetrad verification fails (<code>residual $\neq 0$</code>)	Wrong signature or non-Lorentzian metric	Check <code>g_metric</code> sign convention; supply tetrad manually
Missing LaTeX packages in PDF compile	Incomplete TeX distribution	Install missing packages via MiKTeX package manager or <code>tlmgr install <pkg></code>
<code>NameError: name 'progress' is not defined</code>	Stale <code>.pyc</code> cache	Delete <code>__pycache__/</code> and rerun
Very large PDF / equation overflow	Long symbolic expressions	Normal for abstract-function metrics; expressions wrap automatically

A Complete Configuration Flag Reference

Variable	Type	Default	Purpose
<code>coords</code>	list	<code>[t,r,θ,φ]</code>	Coordinate symbols
<code>dim</code>	int	4	Spacetime dimension
<code>g_metric</code>	Matrix	PG warp	Covariant metric $g_{\mu\nu}$
<code>g_inv_metric</code>	Matrix/None	None	Contravariant metric; <code>None</code> = auto
<code>e_tetrad</code>	Matrix/None	None	Orthonormal tetrad; <code>None</code> = auto
<code>latex_subs</code>	dict	<code>{}</code>	LaTeX string post-replacements
<code>COMPUTE_WEYL</code>	bool	<code>True</code>	Weyl tensor (4D only)
<code>COMPUTE_KRETSCHMANN</code>	bool	<code>True</code>	Kretschmann scalar (slow)
<code>COMPUTE_GEODESICS</code>	bool	<code>True</code>	Geodesic equations
<code>COMPUTE_KILLING</code>	bool	<code>True</code>	Cyclic coordinate detection
<code>COMPUTE_TETRAD</code>	bool	<code>True</code>	Auto-compute tetrad

Variable	Type	Default	Purpose
FAST_MODE	bool	False	Skip Weyl & Kretschmann
USE_PARALLEL	bool	False	Parallel (Linux/Colab only)
N_PARALLEL_JOBS	int	4	Parallel worker count
USE_GPU	bool	False	GPU numerical post-proc. (Colab)
OUTPUT_FILENAME	str	"gr_report"	Base name for .tex/.pdf
AUTHOR_NAME	str	"SymPy GR Engine"	PDF author field
METRIC_NAME	str	(metric name)	PDF title
METRIC_DESCRIPTION	str	(description)	PDF abstract text

B `run_computations()` Parameter Reference

Parameter	Type	Default	Purpose
<code>g_metric</code>	Matrix	—	Covariant metric (required)
<code>coords</code>	list	—	Coordinate symbols (required)
<code>dim</code>	int	—	Spacetime dimension (required)
<code>g_inv_metric</code>	Matrix/None	None	Contravariant metric; None=auto
<code>e_tetrad</code>	Matrix/None	None	User tetrad; None=auto
<code>compute_weyl_flag</code>	bool	True	Compute Weyl tensor
<code>compute_kretschmann_flag</code>	bool	True	Compute K
<code>compute_geodesics_flag</code>	bool	True	Compute geodesics
<code>compute_killing_flag</code>	bool	True	Detect Killing coords
<code>compute_tetrad_flag</code>	bool	True	Auto-compute tetrad
<code>fast_mode</code>	bool	False	Skip Weyl & Kretschmann
<code>compute_parallel</code>	bool	False	Parallel (Colab only)
<code>n_jobs</code>	int	4	Parallel workers (Colab only)

C `gr_numerics.py` Quick Reference (Colab Edition)

Function signature	Description
<code>detect_backend()</code>	Returns 'cupy', 'jax', or 'numpy'
<code>get_backend(name=None)</code>	Returns the array module (np/cp/jnp)

Function signature	Description
<code>lambdify_scalar(expr, coords, backend)</code>	SymPy scalar $\rightarrow$ callable
<code>lambdify_matrix(mat, coords, backend)</code>	SymPy Matrix $\rightarrow$ list of callables
<code>evaluate_scalar_grid(fn, coord_vals)</code>	Evaluate on meshgrid of 1D arrays
<code>evaluate_results_numerical(results, coords, coord_ranges, npts=50)</code>	Lambdify & grid-evaluate 'R_scalar' and 'K'
<code>plot_gr_quantities(num_results, coords, slice_axes=(0,1))</code>	2D heat maps via matplotlib
<code>integrate_geodesic_jax(rhs_fn, y0, tau_span, n_steps)</code>	diffraction/JAX or scipy ODE integration
<code>plot_geodesic(tau_arr, y_arr, coords)</code>	Position vs. τ subplots

D Coordinate LaTeX Map

Symbol name	LaTeX output	Rendered
<code>t</code>	<code>t</code>	t
<code>r</code>	<code>r</code>	r
<code>theta</code>	<code>\theta</code>	θ
<code>phi</code>	<code>\varphi</code>	φ
<code>x</code>	<code>x</code>	x
<code>y</code>	<code>y</code>	y
<code>z</code>	<code>z</code>	z
<code>chi</code>	<code>\chi</code>	χ
<code>psi</code>	<code>\psi</code>	ψ
<code>(any other)</code>	<code>str(symbol)</code>	as-is

These mappings are defined in `gr_tensors._COORD_LATEX_MAP` and are used throughout the PDF report for axis labels and index notation.